Accurate Graph-based Multi-Positive Unlabeled Learning via Disentangled Multi-view Feature Propagation

Junghun Kim
Seoul National University
Seoul, South Korea
bandalg97@snu.ac.kr

Hoyoung Yoon
Seoul National University
Seoul, South Korea
crazy8597@snu.ac.kr

Ka Hyun Park
Seoul National University
Seoul, South Korea
kahyun.park@snu.ac.kr

U Kang
Seoul National University
Seoul, South Korea
ukang@snu.ac.kr

ABSTRACT

How can we classify graph-structured data with labels of only positive classes? Graph-based MPU learning is to train a classifier where only a few nodes of positive classes are labeled and the others remain unlabeled; i.e., negative labels are completely absent. This scenario is deeply connected to real-world problems such as classifying multiple types of cyberattackers (e.g., DDoS, phishing) in network graphs where explicit labels for normal users are unavailable since the undetected cyberattackers disguise themselves as normal users. The main challenge lies in the connections between nodes of different classes, which cause the learned features of positive and unlabeled nodes to become indistinguishable. This issue is particularly severe for negative nodes as there are no observed labels to guide them in MPU settings.

In this paper, we propose D-MVP (Disentangled Multi-view Feature Propagation), an accurate method for graph-based MPU learning. D-MVP disentangles feature propagation into multiple views by assigning distinct weights to each view and aggregating information differently based on these weights. This makes the node classifier learn information that distinguishes between multiple positive classes while simultaneously capturing shared features among them, thereby effectively identifying negative classes that lack these shared characteristics. Extensive experiments on real-world datasets show that D-MVP achieves the best performance.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Information systems** → **Data mining**.

KEYWORDS

graph neural networks, MPU learning

ACM Reference Format:
Junghun Kim, Hoyoung Yoon, Ka Hyun Park, and U Kang. 2025. Accurate Graph-based Multi-Positive Unlabeled Learning via Disentangled Multi-view Feature Propagation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, Toronto, Canada, 11 pages. https://doi.org/10.1145/3711896.3736827

SIGKDD'25, August 3–7, 2025, Toronto, ON, Canada

ACM ISBN 979-8-4007-1454-2/25/08…$15.00
https://doi.org/10.1145/3711896.3736827

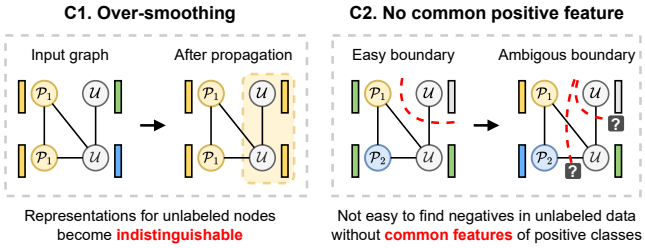

Figure 1: Two main challenges. First, the representations of positive and negative nodes become indistinguishable due to the absence of negative labels to guide the model. Second, if the model focuses only on learning distinctive features between the classes, it struggles to identify negative examples that do not share the common features of positive classes.

1 INTRODUCTION

How can we accurately classify graph-structured data using only positive class labels? Multi-positive Unlabeled (MPU) learning [54, 65] is to train a classifier using only positive class labels when obtaining true negative labels is extremely challenging. This issue frequently arises in practical applications. For instance, in the context of identifying cyberattackers [1] such as those conducting DDoS or malware attacks, attackers often masquerade as normal users. In pandemic classification [34], patients diagnosed with different infectious diseases such as COVID-19, influenza, or pneumonia are labeled as positives, whereas individuals who have not been diagnosed remain unlabeled. The absence of labeled negative nodes complicates the training process for classifiers. MPU learning has demonstrated its effectiveness across various domains including text [41], image [30], tabular, and time series data [47].

Many real-world data are represented as graphs [14, 16, 19, 24, 46, 57, 60]. The cyberattacker and pandemic networks are also graph-structured where nodes represent individuals and edges denote interactions between them. Graph-based PU learning has gained considerable attention [15, 58, 61] for its ability to handle the scenarios without negative labels by leveraging the graph structure. Nevertheless, research on graph-based MPU learning remains underexplored.

Recently, Yoo et al. [59] proposed a belief-based risk estimator for graph-based MPU learning. The main limitation is that their method naively propagates information through the graph, which may cause the representations of positive and negative nodes indistinguishable (over-smoothing [44]). This challenge is even worse for unlabeled nodes since there are no explicit labels to guide the classifier. To address over-smoothing, Wu et al. [53] weaken the

edges between nodes of different classes. However, this approach also weakens the connections among positive classes, limiting the model's ability to capture their shared features. The two main challenges of previous approaches are depicted in Figure 1.

Capturing shared features among positive classes is crucial for detecting negative nodes that lack these features. Simultaneously, learning distinct patterns within positive classes is vital for accurate differentiation. Achieving a balance between these two contrastive objectives is critical in graph-based MPU learning.

In this work, we propose D-MVP (Disentangled Multi-view Feature Propagation), an accurate method for graph-based MPU learning. D-MVP disentangles feature propagation into multiple views by assigning distinct weights to each view and aggregating information differently. This enables independent learning within each view, allowing the model to capture both shared and unique patterns across positive classes. To minimize information overlap between the views, we construct each view using different characteristics such as structure-based and feature-based features. Additionally, D-MVP exploits a contrastive learning strategy to further guide the model in learning shared features among positive classes alongside their distinctive features. Beyond improving performance, analyzing the learned weights across views provides insights into how each class prioritizes different feature views, enhancing interpretability. Our contributions are summarized as:

- **Method.** We propose D-MVP, an accurate method for graph-based MPU learning. D-MVP simultaneously learns shared patterns and distinct features across multiple positive classes by disentangling feature propagation into multiple views.
- **Interpretability.** By analyzing the learned edge weights across multiple views, we reveal how each class prioritizes specific feature views.
- **Experiments.** We conduct extensive experiments and show that D-MVP achieves the best performance.

The remainder of this paper is organized as follows. In Section 2, we review the related literatures and formally define our problem. Section 3 introduces the proposed method, D-MVP, and analyzes its time complexity. Experimental results are provided in Section 4. Finally, we conclude the paper in Section 5. The code and datasets are publicly available at https://github.com/snudatalab/D-MVP.

2 RELATED WORKS

We review related works and present the definition of graph-based MPU learning. Symbols are summarized in Table 6 of Appendix.

2.1 Graph-based PU Learning

PU learning aims to train a binary classifier when only positive (P) and unlabeled (U) examples are available [7, 25]. Traditional PU learning methods identify relatively negative (N) instances in the U examples [13, 17, 27, 31, 39, 49] or regard unlabeled data as weighted positive and negative data simultaneously [6, 18, 22, 38, 42] to transform the problem into a PN (Positive-Negative) learning setup. These methods typically focus on instance-level relationships (e.g., similarity between examples) but overlook the advantages of leveraging graph structures when available.

Graph-based PU learning integrates graph structures into the PU learning, allowing for more informed decision-making regarding unlabeled instances [26, 64]. This paradigm is broadly categorized into two main streams. The first stream focuses on redefining the loss function by utilizing graph structures [36, 55, 58]. The second stream modifies the propagation rules considering the unlabeled instances [52]. However, both approaches are constrained to binary classification setting and cannot easily be adapted to multi-class data, limiting their usability in practical scenarios. D-MVP extends graph-based PU learning to the multi-class setting, broadening its applicability to more complex and realistic applications.

2.2 Graph-based MPU Learning

MPU learning generalizes PU learning to handle multi-class classification problems [54]. The goal is to train a classifier with only a few labeled examples from positive classes, while the rest remain unlabeled [28, 29, 43]. Graph-based MPU learning addresses this problem when explicit relations between examples are represented as a graph. We present the formal definition of the problem as:

Problem 1 (Graph-based MPU Learning). *We are given an undirected graph $G(\mathcal{V}, \mathcal{E})$ and a feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F_{\text{in}}}$ where $\mathcal{V}$ and $\mathcal{E}$ are sets of nodes and edges, respectively. Each i-th row of $\mathbf{X}$ represents the input feature of node i. The node set $\mathcal{V}$ is partitioned into $M+1$ disjoint subsets: $\mathcal{P}_1, \ldots, \mathcal{P}_M$, and $\mathcal{U}$ where M is the number of positive classes, $\mathcal{P}_i$ is the set of labeled nodes of i-th positive class, and $\mathcal{U}$ is the set of unlabeled nodes. The labels of $\mathcal{P}_i$ are represented as a categorical vector $\mathbf{y}_i$. Given G, $\mathbf{X}$, and $\mathbf{y}_i$ for $i = 1, \ldots, M$, graph-based MPU learning is to train a classifier that classifies each node in $\mathcal{U}$ into $M+1$ classes including the negative class.*

Existing MPU learning methods such as URE [54] and AREA [47] combine graph embedding methods like GCN [21] and Node2Vec [10] to utilize the relationships between examples [59]. Yoo et al. [59] propose an expectation-based MPU loss specifically designed for graphs by modeling the given graph as a Markov Network [23]. PU-GNN [55] extends Dist-PU [63] to graph-based MPU (and PU) learning by utilizing graph structures to define distances between nodes. Their main limitation lies in the propagation of information through the given graph under the assumption of homophily, which causes the representations of positive and negative nodes to become indistinguishable. To address the limitation, GPL [53] reduces the strength of edges between nodes of different classes. However, this approach also weakens the connections among positive classes, limiting the model's ability to capture shared features between them. As a result, GPL struggles to effectively distinguish negative examples, which lack the shared features, from the unlabeled ones.

D-MVP does not rely on the assumption of homophily. D-MVP constructs multi-view weighted graphs and propagates information independently within each view, allowing the classifier to capture diverse relational patterns. Moreover, D-MVP maintains strong connections among positive classes by disentangling shared and distinctive features. This ensures that shared patterns among positive classes—essential for distinguishing negatives—are preserved.

2.3 Disentangled Graph Neural Networks

Disentangled representation learning seeks to obtain factorized representations that identify and disentangle the underlying explanatory factors embedded in observed data [3]. Recently, this

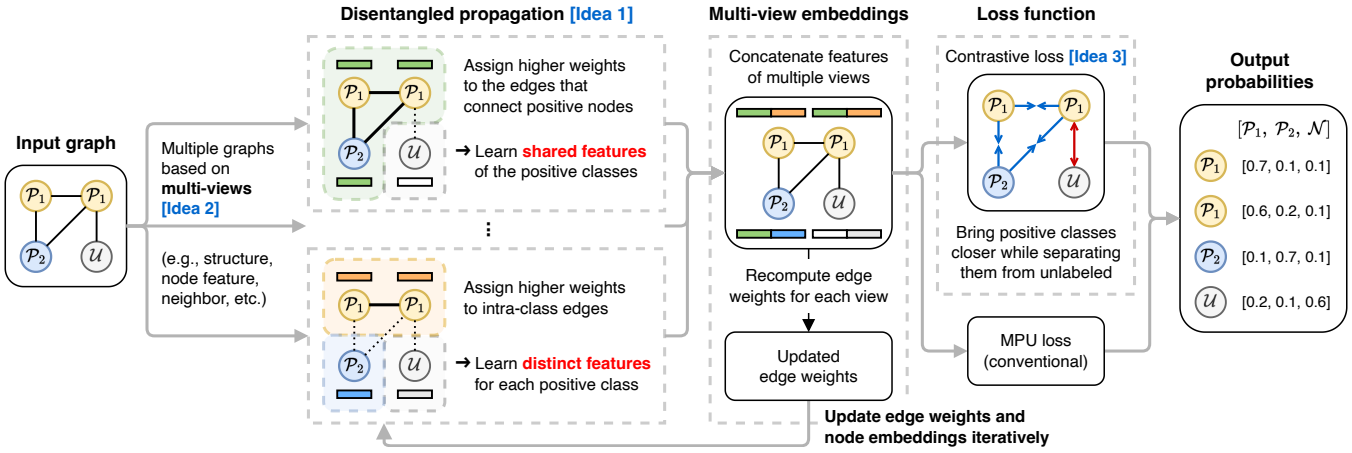

Figure 2: The structure of the proposed D-MVP. First, D-MVP constructs multi-view graphs, each representing different features. Next, D-MVP propagates information independently for each view, allowing the model to simultaneously learn class-specific features and shared features across multiple positive classes. The learned features are concatenated to predict class probabilities and used to update edge weights in subsequent iterations.

approach has garnered increasing attention in the domain of graph-structured data [12, 33, 35, 37]. It has been applied across a range of tasks, including recommendation systems [32, 48], graph classification [50], and graph neural architecture search [62].

Typical disentangled propagation methods differ from D-MVP in two key aspects. First, they generate multiple features using the same process. For example, DisenGCN [35] applies a linear layer to the input node features to produce multiple features. This approach inevitably leads to redundant and overlapping information, hindering the separation of shared and distinctive features among positive classes. In contrast, D-MVP exploits a multi-view representation, ensuring each feature captures distinct aspects of the data.

Second, they often perform multiple rounds of information propagation within each layer (a.k.a. the routing process). This leads to a severe over-smoothing problem, which is particularly problematic in MPU learning. D-MVP addresses this by leveraging global information to construct multi-view features, eliminating the need for repeated propagation.

3 PROPOSED METHOD

We propose D-MVP, an accurate method for graph-based MPU learning. We illustrate the overall process of D-MVP in Figure 2 and Algorithm 1. The main challenges and our approaches are:

(1) **How can we simultaneously learn the shared and distinctive features of positive classes?** We generate multiple versions of node features and construct a separate weighted graph for each feature. Propagation is then performed independently on each graph. This enables the explicit modeling of both shared and class-specific features (Section 3.1).
(2) **How can we reduce the overlapping information across the multiple features?** If the multiple features contain overlapping information, D-MVP redundantly learns the same features. To address this, we propose constructing each feature based on multiple views, which are structural, feature-based, neighbor-based, and statistic-based views, to ensure that each feature serves a distinct role (Section 3.2).
(3) **How can we further guide positive classes to learn shared features?** We propose a contrastive loss that clusters positive classes while separating them from the 'unlabeled' class (Section 3.3).

3.1 Disentangled Feature Propagation

While the positive classes may share structural or semantic similarities that distinguish themselves from negative examples, each positive class also possesses class-specific features. Failing to account for both shared and distinctive features can hinder the ability to draw clear boundaries between positive and negative examples, as well as among positive classes themselves. This dual requirement presents a unique challenge in graph-based MPU learning: the simultaneous need to capture shared features among positive classes while preserving their distinctive characteristics.

To address this challenge, we propose a novel disentangled multi-view feature propagation approach. The key idea is to learn multiple features (*multi-view*) for each node that capture both shared features, which separate positive classes from the negative class, and distinctive features, which further differentiate the positive classes. This is achieved by constructing a weighted graph for each view and independently propagating information within each, a process we refer to as *disentangled propagation*. The edge weights for each view are dynamically and adaptively updated based on the node representations learned by the classifier.

By explicitly modeling both shared and distinctive features, D-MVP ensures more accurate learning of classifier. Since edge weights depend on node representations, improving the classifier's accuracy enhances the reliability of the weights. To iteratively refine these components, D-MVP employs a two-step iterative learning approach: a) train a node classifier using the weighted graphs of multiple views, and b) update the weighted graphs for each view,

Algorithm 1: Overall process of D-MVP.

Input : Graph $G = (\mathcal{V}, \mathcal{E})$, feature matrix $\mathbf{X}$
Output : Best parameters θ^* of the node classifier f_θ

Randomly initialize $\theta = \{\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, ..., \mathbf{W}^{(R)}, \mathbf{W}_{\text{cls}}, \mathbf{b}_{\text{cls}}\}$;
$\theta^* \leftarrow \theta$;
$e_{uv}^{(r)} \leftarrow 1, \forall (u,v) \in \mathcal{E}, r = 1, ..., R$; // Initial edge weight
$l^{\text{new}} \leftarrow \infty$; // Initial loss value
repeat
  $l^{\text{old}} \leftarrow l^{\text{new}}$;
  $z_u \leftarrow \{\mathbf{z}_u^{(1)}, \mathbf{z}_u^{(2)}, ..., \mathbf{z}_u^{(R)}\}, \forall u \in \mathcal{V}$; // Section 3.2
  repeat
    for *view* $r = 1, ..., R$ **do**
      $\mathbf{h}_u^{(r)} \leftarrow \sigma(\frac{1}{d_u}\mathbf{W}^{(r)}\mathbf{z}_u^{(r)} + \sum_{(u,v)\in\mathcal{E}} \frac{e_{uv}^{(r)}}{\sqrt{d'_u d'_v}}\mathbf{W}^{(r)}\mathbf{z}_v^{(r)} + \mathbf{b}^{(r)})$ $\forall u \in \mathcal{V}$; // Eq. (1)
    end
    $\mathbf{h}_u \leftarrow [\mathbf{h}_u^{(1)} \mid \cdots \mid \mathbf{h}_u^{(R)}], \ \forall u \in \mathcal{V}$; // Vertical concat.
    $\hat{\mathbf{y}}_u \leftarrow \text{softmax}(\mathbf{W}_{\text{cls}}\mathbf{h}_u + \mathbf{b}_{\text{cls}}), \ \forall u \in \mathcal{V}$;
    $\theta \leftarrow \theta - \nabla_\theta \mathcal{L}(\theta)$; // Eq. (8)
    if $\mathcal{L}(\theta) < \mathcal{L}(\theta^*)$ **then**
      $\theta^* \leftarrow \theta$; // Update the best parameters
    end
  until *the maximum number of epochs is reached*;
  $e_{uv}^{(r)} \leftarrow \frac{\exp(\mathbf{h}_u^{(r)\top}\mathbf{h}_v^{(r)})}{\sum_{i=1}^{R}\exp(\mathbf{h}_u^{(i)\top}\mathbf{h}_v^{(i)})}, \forall (u,v) \in \mathcal{E}, r = 1, ..., R$; // Eq. (2)
  $l^{\text{new}} \leftarrow \mathcal{L}(\theta^*)$;
until $l^{\text{new}} \geq l^{\text{old}}$;

which serve as evidence for training the classifier in the next iteration. The following Sections 3.1.1 and 3.1.2 explain these steps of D-MVP, with a focus on the disentangled feature propagation.

3.1.1 Training node classifier. D-MVP first generates multi-view features that capture various aspects of the given graph $G(\mathcal{V}, \mathcal{E})$ and its input feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F_{\text{in}}}$. Here, $\mathcal{V}$ and $\mathcal{E}$ are the sets of nodes and edges, respectively, $N = |\mathcal{V}|$ is the number of nodes, and F_{in} is the dimension of input node features. For each node u, the multi-view features are represented as $z_u = \{\mathbf{z}_u^{(1)}, \mathbf{z}_u^{(2)}, ..., \mathbf{z}_u^{(R)}\}$, where R is the number of views. To ensure that each view captures distinct and meaningful aspects of the data, it is essential to minimize overlapping information between views. The detailed process for generating multi-view features is provided in Section 3.2.

Once the multi-view features are generated, a separate weighted graph is constructed for each view to enable independent propagation. Given z_u and $\{z_v \mid (u,v) \in \mathcal{E}\}$, the disentangled propagation produces a node embedding $\mathbf{h}_u = [\mathbf{h}_u^{(1)} \mid \cdots \mid \mathbf{h}_u^{(R)}] \in \mathbb{R}^{R \cdot F_{\text{out}}}$ where $\mathbf{h}_u^{(r)} \in \mathbb{R}^{F_{\text{out}}}$ is the propagated result of r-th feature view, with $\mid$ denoting vertical concatenation. The node embeddings are represented as a matrix $\mathbf{H} \in \mathbb{R}^{N \times R \cdot F_{\text{out}}}$ where the u-th row is $\mathbf{h}_u^\top$.

The edge weight $e_{uv}^{(r)}$ for an edge $(u,v) \in \mathcal{E}$ in the r-th view determines the influence of neighboring nodes v on node u during the propagation in the r-th view. In the first iteration, all weights $e_{uv}^{(r)}$ are initialized to 1, as there is no prior information available to measure the similarities between the features in each view. After the first iteration, weights $e_{uv}^{(r)}$ are updated based on the learned node embeddings $\mathbf{h}_u$ and $\mathbf{h}_v$.

The key idea is to independently update the edge weights $e_{uv}^{(r)}$ for each view while ensuring that their sum across all views remains 1: $\sum_{r=1}^{R} e_{uv}^{(r)} = 1$. This ensures the overall importance of each edge is preserved while allowing each view to emphasize distinct aspects of the graph structure and features. A detailed explanation of the process for computing the edge weights is provided in Section 3.1.2.

Then the disentangled propagation of node u in view r is expressed as follows:

$$\mathbf{h}_u^{(r)} = \sigma\left(\frac{1}{d_u}\mathbf{W}^{(r)}\mathbf{z}_u^{(r)} + \sum_{(u,v)\in\mathcal{E}} \frac{e_{uv}^{(r)}}{\sqrt{d'_u d'_v}}\mathbf{W}^{(r)}\mathbf{z}_v^{(r)} + \mathbf{b}^{(r)}\right) \quad (1)$$

where $\mathbf{z}_v^{(r)} \in \mathbb{R}^{F_{\text{in}}}$ is the r-th view feature for node v, $\mathbf{W}^{(r)} \in \mathbb{R}^{F_{\text{out}} \times F_{\text{in}}}$ is the learnable embedding matrix for the r-th view, $\mathbf{b}^{(r)} \in \mathbb{R}^{F_{\text{out}}}$ is the bias, $\sigma(\cdot)$ is the ReLU activation function, d_u is the degree of node u, and d'_u is the degree of node u calculated using the weighted graph.

The embeddings $\mathbf{h}_u$ are used to predict class probabilities through a linear layer. For the objective function, any MPU loss can be used. To explicitly learn the shared features among positive classes, we further propose a contrastive loss, which is detailed in Section 3.3.

3.1.2 Updating weights of multi-view graphs. Larger edge weights $e_{uv}^{(r)}$ encourage the r-th feature embeddings of nodes u and v to become similar, while smaller $e_{uv}^{(r)}$ make them less similar. This property highlights the ability of edge weights to control the similarity of node embeddings for each feature view.

A trained node classifier generates R differently-viewed feature embeddings $\mathbf{h}_u^{(1)}, ..., \mathbf{h}_u^{(R)}$ for each node u. Since the feature similarities in each view represent the degree of shared information within that view, we update the weights of the multi-view graphs based on the similarities. This ensures that the graph weights align with the underlying structure of the learned feature space, further enhancing the embedding quality in the next iteration.

The main intuition behind updating $e_{uv}^{(r)}$ is to allow each view to focus independently on distinct aspects of the graph structure and node features while preserving the overall importance of each edge. To achieve this, the edge weights for all views are updated separately, ensuring that their sum remains 1 for every edge. This is done by applying the Softmax function along the view axis, normalizing the similarities. As a result, the edge weights across all views form a valid probability distribution, satisfying $\sum_{r=1}^{R} e_{uv}^{(r)} = 1$.

We first normalize the embedding matrix $\mathbf{H}$ by applying Gaussian normalization, which standardizes each column to have zero mean and unit variance. This ensures consistent scaling across multiple views. Given the learned $\mathbf{h}_u = [\mathbf{h}_u^{(1)} \mid \cdots \mid \mathbf{h}_u^{(R)}]$ and $\mathbf{h}_v$ for $(u,v) \in \mathcal{E}$, we update the edge weights $e_{uv}^{(r)}$ for each r-th view as follows:

$$e_{uv}^{(r)} = \frac{\exp(\mathbf{h}_u^{(r)\top}\mathbf{h}_v^{(r)})}{\sum_{i=1}^{R}\exp(\mathbf{h}_u^{(i)\top}\mathbf{h}_v^{(i)})}. \quad (2)$$

The updated edges weights are used for the disentangled multi-view propagation (Equation (1)) in the subsequent iteration.

These adaptive edge weights not only refine the feature embeddings but also offer interpretability. Analyzing their distributions reveals how D-MVP prioritizes different feature views for each class, providing insights into its decision-making process.

3.2 Multi-view Feature Construction

We aim to learn each embedding $\mathbf{h}_u^{(r)}$ of the r-th view of node u so that it serves a distinct role from other views, such as capturing shared or distinctive features of positive classes. Minimal overlap across views ensures each feature serves a distinct role, enhancing the model's ability to identify negative nodes and classify positives.

To address this, we propose constructing multi-view features $z_u = \{\mathbf{z}_u^{(1)}, ..., \mathbf{z}_u^{(R)}\}$ for each node u by capturing diverse aspects of the given graph and input features. Specifically, we generate four distinct feature sets based on different views: structural feature ($\mathbf{z}_u^{(1)}$), static feature ($\mathbf{z}_u^{(2)}$), node feature ($\mathbf{z}_u^{(3)}$), and neighbor feature ($\mathbf{z}_u^{(4)}$). Each of these features is propagated through a distinct weighted graphs as described in Equation (1). We also incorporate an additional feature, referred to as the MLP feature ($\mathbf{z}_u^{(5)}$). Unlike the other four features, the MLP feature is not involved in the propagation process, allowing it to retain the original and unpropagated state of the node. In the following, we detail the process of generating $z_u = \{\mathbf{z}_u^{(1)}, ..., \mathbf{z}_u^{(5)}\}$ for each node u.

Structural feature. The structural features are derived from the adjacency matrix $\mathbf{A} \in \mathbb{R}^{N\times N}$ to capture global structural patterns. However, directly using $\mathbf{A}$ as a feature is not effective due to its high dimensionality and sparsity. Thus, we adopt a low-rank approximation of $\mathbf{A}$ using SVD to extract compact and informative structural features. Specifically, we decompose $\mathbf{A}$ into $\mathbf{U}_k \in \mathbb{R}^{N\times k}$, $\Sigma_k \in \mathbb{R}^{k\times k}$, and $\mathbf{V}_k \in \mathbb{R}^{N\times k}$, where Σ_k is a diagonal matrix containing the top-k singular values and $\mathbf{U}_k$ is the corresponding left singular vectors. Then the structural feature $\mathbf{z}_u^{(1)}$ for node u is:

$$\mathbf{z}_u^{(1)\top} = \mathbf{U}_k(u,:)\,\Sigma_\mathbf{k} \tag{3}$$

where $\mathbf{U}_k(u,:)$ is the u-th row of $\mathbf{U}_k$. The parameter k is chosen such that $|\Sigma_k|_F^2/|\mathbf{A}|_F^2 \geq 0.99$, ensuring the approximation preserves key structural information while keeping the features compact.

Static feature. The static features are computed to capture graph-theoretic properties of each node. The features include the degree, clustering coefficient [51], PageRank [40], betweenness centrality [9], closeness centrality [2], eigenvector centrality [5], average shortest path length, k-core number [45], and eccentricity. For a node u, the static feature vector $\mathbf{z}_u^{(2)}$ is composed of the values of these properties for the node.

Node and Neighbor features. The node features represent the intrinsic attributes of each node, while the neighbor features capture local relational information. For a node u, the node feature $\mathbf{z}_u^{(3)}$ is the u-th row of the input feature matrix $\mathbf{X}$. The neighbor feature is obtained by aggregating the input features of a node's neighbors to reflect their influence on the node. The neighbor feature $\mathbf{z}_u^{(4)}$ of node u is as follows:

$$\mathbf{z}_u^{(4)\top} = \sum\nolimits_{(u,v)\in\mathcal{E}} \frac{1}{d_u}\mathbf{X}(v,:) \tag{4}$$

where d_u is the degree of node u.

MLP feature. Similar to the node feature $\mathbf{z}_u^{(3)}$, the MLP feature $\mathbf{z}_u^{(5)}$ is also defined as the u-th row of $\mathbf{X}$. The key difference lies in their involvement in the propagation process: while $\mathbf{z}_u^{(3)}$ is propagated through its own weighted graph as described in Equation (1), $\mathbf{z}_u^{(5)}$ is directly transformed by a linear layer without being propagated. This can be interpreted as setting all edge weights $e_{uv}^{(5)}$ in the 5-th view to zero. As a result, the embedding $\mathbf{h}_u^{(5)}$ is derived solely from the node's own feature $\mathbf{z}_u^{(5)}$, whereas $\mathbf{h}_u^{(3)}$ incorporates the features $\mathbf{z}_v^{(3)}$ from neighboring nodes v. This allows the MLP feature to retain the raw information that might otherwise be diluted through the propagation process, complementing the other feature views.

3.3 Contrastive Loss

In graph-based MPU learning, capturing shared features among positive classes that separate them from the negative class is crucial. D-MVP achieves this by the disentangled propagation of multi-view features as described in Sections 3.1-3.2. To further enhance this, we propose a contrastive loss that operates on hidden embeddings $\mathbf{h}_u \in \mathbb{R}^{5\cdot F_{\text{out}}}$, constructed by vertically concatenating embeddings from all five views: $\mathbf{h}_u = [\mathbf{h}_u^{(1)} \mid \cdots \mid \mathbf{h}_u^{(5)}]$. This contrastive loss encourages embeddings of nodes from positive classes to cluster while ensuring sufficient separation from the negative class.

The contrastive loss $\mathcal{L}_{\text{cont}}$ is defined as the average of the losses for positive and negative pairs:

$$\mathcal{L}_{\text{cont}} = (\mathcal{L}_{\text{pos}} + \mathcal{L}_{\text{neg}})/2 \tag{5}$$

where $\mathcal{L}_{\text{pos}}$ measures the distances between nodes of positive classes and $\mathcal{L}_{\text{neg}}$ computes the negative distances between nodes of positive and negative classes.

The positive pair loss $\mathcal{L}_{\text{pos}}$ is defined as:

$$\mathcal{L}_{\text{pos}} = \frac{1}{|\mathcal{E}_{\text{pos}}|}\sum\nolimits_{(u,v)\in\mathcal{E}_{\text{pos}}} \|\mathbf{h}_u - \mathbf{h}_v\|_2^2, \tag{6}$$

where $\mathcal{E}_{\text{pos}}$ is the set of edges connecting nodes in positive classes, including intra-positive (within the same positive class) and inter-positive (between different positive classes) edges. This term encourages embeddings of nodes in positive classes to cluster together, thereby capturing their shared features.

The negative pair loss $\mathcal{L}_{\text{neg}}$ penalizes embeddings of positive and negative classes that are closer than a fixed margin of 1:

$$\mathcal{L}_{\text{neg}} = \frac{1}{|\mathcal{E}_{\text{neg}}|}\sum\nolimits_{(u,v)\in\mathcal{E}_{\text{neg}}} \max(0, 1 - \|\mathbf{h}_u - \mathbf{h}_v\|_2^2), \tag{7}$$

where $\mathcal{E}_{\text{neg}}$ is the set of edges connecting positive and unlabeled nodes. This term ensures the separation between positive and negative embeddings. To balance the contributions of positive and negative pairs, the number of negative pairs $|\mathcal{E}_{\text{neg}}|$ is equalized with the number of positive pairs $|\mathcal{E}_{\text{pos}}|$ by random sampling.

The contrastive loss $\mathcal{L}_{\text{cont}}$ is incorporated in addition to any conventional MPU loss $\mathcal{L}_{\text{mpu}}$. Thus, the final objective function $\mathcal{L}$ of D-MVP is as follows:

$$\mathcal{L} = \mathcal{L}_{\text{mpu}} + \mathcal{L}_{\text{cont}}. \tag{8}$$

In the experiments, we use the loss of GRAB [59] as $\mathcal{L}_{\text{mpu}}$ due to its robustness and high accuracy, which is defined as

$$\mathcal{L}_{\text{mpu}} = \frac{1}{\sum_{m=1}^{M}|\mathcal{P}_m|}\sum_{m=1}^{M}\sum_{i\in\mathcal{P}_m} l(\mathbf{y}_i, \hat{\mathbf{y}}_i) + \frac{1}{|\mathcal{U}|}\sum_{j\in\mathcal{U}} l(\mathbf{b}_j, \hat{\mathbf{y}}_j) \tag{9}$$

where $\mathbf{y}_i \in \mathbb{R}^{M+1}$ and $\hat{\mathbf{y}}_i \in \mathbb{R}^{M+1}$ denote the one-hot label and the prediction probability of node i, respectively, and $l(\cdot)$ is the cross-entropy loss. $\mathbf{b}_j \in \mathbb{R}^{M+1}$ is a probability vector representing the belief, computed using loopy belief propagation. The belief vector $\mathbf{b}_j$ represents approximate marginal probabilities of nodes obtained

by propagating class priors through the graph, and serves as soft pseudo labels for unlabeled nodes.

By minimizing both the contrastive loss and the MPU loss, our approach reinforces the preservation of shared features among positive classes while maintaining their distinctiveness.

3.4 Time and Space Complexities

D-MVP is scalable to large graphs, as its computational complexity scales linearly with the number of nodes and edges. In the following Lemmas 1 and 2, we analyze the time and space complexity of D-MVP. We assume D-MVP with r layers where each view has d-dimensional features across all layers.

Lemma 1. *The time complexity of D-MVP is*

$$O(Tn_i drR(|\mathcal{E}| + |\mathcal{V}|d)), \tag{10}$$

where $\mathcal{V}$ and $\mathcal{E}$ are sets of nodes and edges, respectively, and R is the number of views. T is the total number of iterations and n_i is the number of training steps per iteration.

Proof. Each iteration of D-MVP consists of two main steps: a) updating edge weights and b) training the classifier. For each view, we update edge weights based on the similarity between each feature. Since we perform this across R views and $|\mathcal{E}|$ edges, this step takes $O(dR|\mathcal{E}|)$ time. The complexity of updating the classifier for each epoch is $O(drR(|\mathcal{E}| + |\mathcal{V}|d))$ [58]. Thus, the total time complexity of D-MVP is $O(Tn_i drR(|\mathcal{E}| + |\mathcal{V}|d))$. □

Lemma 2. *The space complexity of D-MVP is*

$$O(R(d^2r + dr|\mathcal{V}| + |\mathcal{E}|)), \tag{11}$$

where $\mathcal{V}$ and $\mathcal{E}$ are sets of nodes and edges, respectively, and R is the number of views.

Proof. The training of D-MVP requires storing the node features and weight matrices at all layers. Thus, the total space complexity of D-MVP is $O(R(d^2r + dr|\mathcal{V}| + |\mathcal{E}|))$. □

4 EXPERIMENTS

We perform experiments to answer the following questions:

- **Q1. Classification accuracy (Section 4.2).** Does D-MVP show superior performance than the baselines in graph-based MPU learning?
- **Q2. Effect of iterative learning (Section 4.3).** How does the performance of D-MVP change over iterations?
- **Q3. Applying D-MVP to other baselines (Section 4.4).** Does applying the propagation method of D-MVP to other MPU-learning losses improve the accuracy?
- **Q4. Discussion on edge weights (Section 4.5).** Do the edge weights reveal shared patterns in positive classes that distinguish themselves from negatives while preserving their distinctiveness?
- **Q5. Accuracy with noisy data (Section 4.6).** Is D-MVP robust to noisy graphs with randomly added edges?
- **Q6. Training efficiency (Section 4.7).** How efficient is D-MVP in terms of training time per epoch compared to other models?

Table 1: Summary of datasets.

Name	Nodes	Edges	Feat.	Pos.1	Pos.2	Pos.3	Neg.
Cora[1]	2,708	10,556	1,433	818	426	418	1,046
Cora-ML[2]	2,995	16,316	2,879	857	452	442	1,244
CiteSeer[1]	3,327	9,104	3,703	701	668	596	1,362
CiteSeer-full[2]	4,230	10,674	602	831	778	740	1,881
Chameleon[3]	2,277	62,742	2,325	521	460	456	840

[1] https://github.com/kimiyoung/planetoid
[2] https://github.com/abojchevski/graph2gauss
[3] https://github.com/benedekrozemberczki/MUSAE

4.1 Experimental Settings

4.1.1 Datasets. We use four publicly available real-world datasets, which are summarized in Table 1. Cora, Cora-ML, CiteSeer, and CiteSeer-full are citation networks [4, 56] widely used in node classification tasks. In these networks, nodes represent scientific publications, which are categorized based on research areas, and the features are represented as bag-of-words vectors derived from their textual content. Chameleon is a Wikipedia network where nodes correspond to web pages and edges represent hyperlinks between them. Node features include keywords or informative nouns extracted from the content of the web pages.

For MPU learning, datasets with n positive labels are constructed by selecting the n classes with the largest number of nodes as positive classes, while the remaining nodes are treated as negative. In our experiments, n is set to 3 following [59], and the resulting numbers of positive and negative nodes are reported in Table 1 along with other dataset statistics such as the number of edges.

4.1.2 Baselines. We compare our method against several established baselines. These include: 1) Naive GCN [21] and MLP [11], which are standard benchmarks for tasks involving graph-structured data, 2) URE [54] and AREA [47], two popular methods originally designed for MPU learning in i.i.d. data and implemented here with MLP and GCN as the backbone model, 3) GRAB [59] and PU-GNN [55], the current state-of-the-art approaches for graph-based MPU learning, and 4) GPL [53], a recently proposed method tailored for graph-based PU learning. For GPL, we extend their PU loss function to an MPU loss function to suit the multi-class setting. Additional details about these baselines are provided in Appendix A.2.

4.1.3 Evaluation. We evaluate the performance of D-MVP and the baselines in classifying unlabeled nodes. For each dataset, we use 20% of the nodes from each positive class as the observed positive set, denoted as $\mathcal{P}_i$ where i represents the positive class index. The remaining nodes from each positive class are treated as unobserved positive nodes, denoted as $\mathcal{P}_i^{\text{test}}$. All nodes belonging to negative classes are treated as unobserved and are denoted by $\mathcal{N}^{\text{test}}$, as the MPU learning setting assumes a positive-unlabeled configuration. Our goal is to predict the label of each node in $\mathcal{U} = \bigcup_i \mathcal{P}_i^{\text{test}} \cup \mathcal{N}^{\text{test}}$ by training a classifier using labeled nodes $\bigcup_i \mathcal{P}_i$ and unlabeled nodes $\mathcal{U}$; all nodes are accessible during training, but only the labels of $\bigcup_i \mathcal{P}_i$ are available.

Many baselines assume that the class prior probabilities, which represent the true distribution of instances across multiple positive

Table 2: Classification performance of D-MVP and baselines in terms of macro-F1, accuracy, and KL divergence. We conduct two types of experiments: 1) with the prior unknown for all methods, and 2) with the prior known for all methods except GRAB, PU-GNN, GPL, and the proposed method, which do not require the prior to be known in advance. The best and second-best performances are highlighted in green and yellow. D-MVP consistently achieves the best performance.

Unknown class probabilities

Models	Cora			Cora-ML			CiteSeer			CiteSeer-full			Chameleon		
	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)
MLP	16.4 ± 0.0	48.3 ± 0.0	6.8919	17.4 ± 0.2	52.0 ± 0.1	4.2230	17.1 ± 0.1	51.2 ± 0.0	5.6568	24.1 ± 0.9	57.5 ± 0.3	1.1111	17.1 ± 0.2	46.5 ± 0.1	3.9556
GCN	19.8 ± 0.5	49.5 ± 0.2	1.5636	21.4 ± 0.4	53.2 ± 0.1	1.4065	21.6 ± 1.0	52.4 ± 0.3	1.2530	28.7 ± 0.6	58.9 ± 0.2	0.8473	18.1 ± 0.3	46.8 ± 0.1	1.8561
URE+MLP	16.6 ± 0.1	48.4 ± 0.0	4.2807	17.4 ± 0.0	52.1 ± 0.0	4.8231	17.2 ± 0.1	51.2 ± 0.0	4.2346	23.6 ± 1.3	57.2 ± 0.4	1.4480	16.8 ± 0.2	46.3 ± 0.1	2.2128
URE+GCN	17.7 ± 0.7	49.0 ± 0.3	4.8593	18.1 ± 0.2	52.2 ± 0.1	5.6120	18.5 ± 0.7	51.6 ± 0.2	4.8741	22.9 ± 0.5	57.1 ± 0.2	4.9060	16.1 ± 0.1	46.2 ± 0.0	5.5656
AREA+MLP	27.7 ± 1.4	52.1 ± 0.5	0.6952	30.0 ± 2.3	56.8 ± 1.1	0.8849	27.3 ± 1.6	53.6 ± 1.0	0.7706	58.2 ± 3.3	69.9 ± 1.7	0.0789	29.4 ± 1.7	50.6 ± 0.9	0.8270
AREA+GCN	57.1 ± 4.2	69.0 ± 1.3	0.1602	39.0 ± 2.6	62.8 ± 1.4	1.3419	56.9 ± 1.6	65.4 ± 0.8	0.0418	83.8 ± 1.4	87.7 ± 0.7	0.0093	37.4 ± 4.0	37.3 ± 3.3	0.3564
GRAB	84.8 ± 1.2	84.9 ± 1.3	0.0166	86.3 ± 0.3	86.5 ± 0.2	0.0143	59.5 ± 15.3	59.8 ± 16.4	3.5184	84.2 ± 1.0	87.2 ± 0.8	0.0256	38.6 ± 0.2	50.0 ± 0.1	7.8324
PU-GNN	80.6 ± 1.2	83.3 ± 1.0	0.0071	85.2 ± 0.6	86.7 ± 0.8	0.0009	63.7 ± 0.8	70.4 ± 0.6	0.0494	79.7 ± 0.5	84.8 ± 0.4	0.0133	38.8 ± 0.4	39.0 ± 0.4	1.6018
GPL	52.9 ± 1.5	60.7 ± 1.3	0.0067	35.2 ± 1.0	46.9 ± 3.3	0.2530	58.9 ± 0.1	64.1 ± 0.1	0.0040	77.9 ± 0.2	81.7 ± 0.1	0.0110	39.1 ± 0.1	50.0 ± 0.1	0.5198
D-MVP (Ours)	87.8 ± 0.2	88.1 ± 0.2	0.0030	88.6 ± 0.2	89.4 ± 0.2	0.0008	69.9 ± 0.1	73.6 ± 0.1	0.0183	86.7 ± 1.0	87.9 ± 1.0	0.0148	56.3 ± 5.0	61.2 ± 6.7	0.1456

Known class probabilities

Models	Cora			Cora-ML			CiteSeer			CiteSeer-full			Chameleon		
	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)
MLP	16.4 ± 0.0	48.3 ± 0.0	6.8919	17.4 ± 0.2	52.0 ± 0.1	4.2230	17.1 ± 0.1	51.2 ± 0.0	5.6568	24.1 ± 0.9	57.5 ± 0.3	1.1111	17.1 ± 0.2	46.5 ± 0.1	3.9556
GCN	20.2 ± 1.0	49.6 ± 0.4	1.5135	21.4 ± 0.4	53.2 ± 0.1	1.4068	22.1 ± 0.8	52.5 ± 0.2	1.1783	28.5 ± 0.6	58.8 ± 0.2	0.8541	18.2 ± 0.2	46.8 ± 0.1	1.8402
URE+MLP	21.0 ± 0.4	50.1 ± 0.1	2.8443	19.4 ± 0.6	52.7 ± 0.1	2.8800	21.7 ± 0.2	52.6 ± 0.1	1.2734	47.6 ± 0.7	65.9 ± 0.2	0.3178	23.3 ± 3.7	48.6 ± 1.6	1.5734
URE+GCN	38.0 ± 3.0	58.3 ± 0.9	1.7047	27.6 ± 1.6	57.7 ± 0.4	3.6702	37.1 ± 1.7	57.8 ± 0.6	0.5046	68.1 ± 1.3	76.9 ± 0.7	0.1354	45.6 ± 1.0	60.1 ± 0.6	3.2446
AREA+MLP	31.0 ± 1.2	52.7 ± 0.6	0.4914	29.9 ± 3.9	46.0 ± 17.0	2.4357	27.4 ± 4.2	44.2 ± 15.2	2.3986	58.8 ± 4.3	69.9 ± 2.5	0.0519	29.4 ± 2.5	49.9 ± 1.3	0.6579
AREA+GCN	50.7 ± 6.0	66.0 ± 2.6	0.2171	29.8 ± 4.7	43.7 ± 8.0	1.6840	55.0 ± 1.2	64.1 ± 1.1	0.0444	74.8 ± 3.2	82.7 ± 2.0	0.0423	29.7 ± 2.6	30.7 ± 2.0	0.5125
GRAB	84.8 ± 1.2	84.9 ± 1.3	0.0166	86.3 ± 0.3	86.5 ± 0.2	0.0143	59.5 ± 15.3	59.8 ± 16.4	3.5184	84.2 ± 1.0	87.2 ± 0.8	0.0256	38.6 ± 0.2	39.0 ± 0.2	7.8324
PU-GNN	80.6 ± 1.2	83.3 ± 1.0	0.0071	85.2 ± 0.6	86.7 ± 0.8	0.0009	63.7 ± 0.8	70.4 ± 0.6	0.0494	79.7 ± 0.5	84.8 ± 0.4	0.0133	38.8 ± 0.4	39.0 ± 0.4	1.6018
GPL	52.9 ± 1.5	60.7 ± 1.3	0.0067	35.2 ± 1.0	46.9 ± 3.3	0.2530	58.9 ± 0.1	64.1 ± 0.1	0.0040	77.9 ± 0.2	81.7 ± 0.1	0.0110	39.1 ± 0.1	50.0 ± 0.1	0.5198
D-MVP (Ours)	87.8 ± 0.2	88.1 ± 0.2	0.0030	88.6 ± 0.2	89.4 ± 0.2	0.0008	69.9 ± 0.1	73.6 ± 0.1	0.0183	86.7 ± 1.0	87.9 ± 1.0	0.0148	56.3 ± 5.0	61.2 ± 6.7	0.1456

classes and the negative class, are known beforehand. Thus, we conduct two types of experiments for the baselines: 1) with the class prior probabilities and 2) without them.

We primarily use macro-F1 score and classification accuracy as the evaluation metrics, while also reporting the KL divergence between the true class prior and the estimated prior for a more comprehensive assessment. We conduct ten independent experiments using random seeds. To ensure a more robust comparison between methods, we exclude the two highest and lowest values, and report the mean performance along with its standard deviation.

4.2 Accuracy (Q1)

We compare the MPU learning performance of D-MVP and the baselines in Table 2, with and without the class prior probabilities. GRAB, PU-GNN, GPL, and the proposed D-MVP do not require the class prior to be known beforehand and therefore do not utilize it in either case. Note that D-MVP consistently outperforms the baselines in terms of macro-F1 scores and classification accuracy.

GRAB [59] and GPL [53] also exploit iterative training strategy to gradually improve the performance. However, a key difference between these methods and D-MVP is that they are restricted to learning only one type of feature—either common or distinctive—whereas the proposed method effectively learns both. GRAB propagates features through the input graph under the assumption of homophily, which causes over-smoothing and limits the learning of distinctive features in positive classes. GPL disconnects edges between different classes, which prevents the classifier from learning the shared features among positive classes.

Notably, D-MVP demonstrates superior performance compared to the iterative methods GRAB and GPL. These results highlight the importance of simultaneously learning both the shared and distinctive features of positive classes across multiple views to achieve high performance in graph-based MPU learning.

4.3 Effect of Iterative Learning (Q2)

D-MVP iteratively refines the edge weights assigned to each feature view and re-trains the classifier based on the updated representations. To evaluate the impact of this refinement on performance, we analyze how the classification performance evolves over iterations. The results are presented in Figure 3.

The top three plots illustrate the changes in macro-F1 and classification accuracy over iterations. As the iterations progress, both metrics show consistent improvement, demonstrating the positive effect of refining the weights for each feature view.

The bottom three plots compare the estimated class prior probabilities with the true ones for each of the three positive classes, a critical consideration in imbalanced data scenarios. The estimated (or true) class prior probability of a positive class $\mathcal{P}_i$ is the proportion of unlabeled instances estimated to (or actually) belong to $\mathcal{P}_i$. The results reveal that the estimated prior gradually aligns with the true prior, while macro-F1 and accuracy also converge alongside it. In the CiteSeer, the estimated prior appears to continue increasing when iterations are forcibly extended. However, the iterations of D-MVP successfully terminate at iteration 2 by comparing the loss of the previous iteration with the current loss, effectively achieving stability and preventing unnecessary over-iterations.

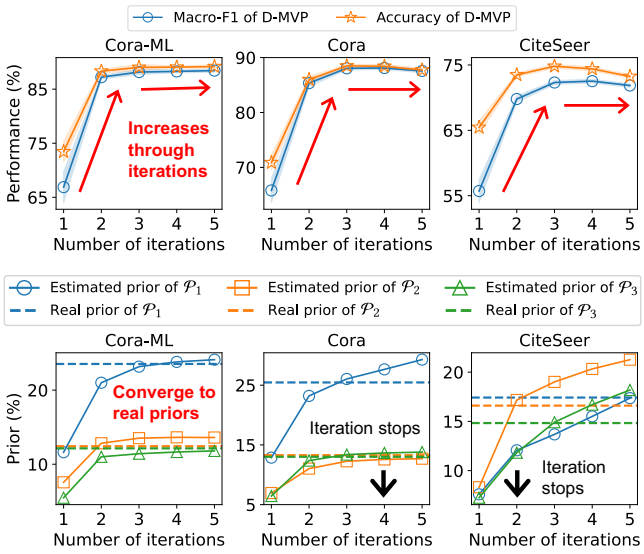

Figure 3: Performance change of D-MVP across the iterations. The top three plots demonstrate that the accuracy of D-MVP improves progressively through iterations. The bottom three plots show that D-MVP increasingly aligns its predicted class priors with the true ones, a critical aspect in imbalanced data.

4.4 Applying to Other Baselines (Q3)

D-MVP proposes a propagation technique and a contrastive loss that can be integrated with conventional MPU loss functions. In previous experiments, the belief-based loss of GRAB is used as the conventional MPU loss term $\mathcal{L}_{\text{mpu}}$ in Equation (8). To showcase its versatility, we integrate D-MVP with other MPU learning losses such as URE and AREA. Since the URE loss strongly depends on the known prior probabilities as shown in Table 2, we assume the prior is known for both URE and URE+D-MVP. For AREA and AREA+D-MVP, we adopt the more realistic assumption that the prior is unknown, following [58].

The results are summarized in Table 3. Note that D-MVP consistently improves the performance of these losses, demonstrating its broad applicability across various MPU loss methods.

4.5 Discussion on Edge Weights (Q4)

D-MVP captures shared features among positive classes that differentiate them from the negative class, while also learning class-specific distinctive features within the positive classes. To evaluate whether the representations effectively disentangle the shared and distinctive components among positive classes, we analyze the learned representations in detail. Since the edge weights for each view are computed by measuring the similarities between the representations in each view, we analyze these learned edge weights. This enables us to understand how D-MVP assigns importance to different feature views across classes, offering deeper insights into its decision-making process. From this analysis, we present two key observations.

First, we study how the edge weights of positive-positive (Pos-Pos) edges differ from those of positive-negative (Pos-Neg) edges,

Table 3: Performance improvement of baselines integrated with D-MVP. Bold numbers indicate the best performance. Note that D-MVP enhances the performance of baselines in most cases, demonstrating its broad applicability.

Datasets		URE	AREA	URE+D-MVP	AREA+D-MVP
Cora	F1 (↑)	**38.0 ± 3.0**	57.1 ± 4.2	36.5 ± 3.4 (↓ 3.9%)	**65.6 ± 2.6** (↑ 14.9%)
	Acc. (↑)	**58.3 ± 0.9**	69.0 ± 1.3	57.1 ± 1.0 (↓ 2.1%)	**73.5 ± 1.5** (↑ 6.5%)
	KL. (↓)	**1.7047**	0.1602	1.7667 (↑ 3.6%)	**0.1122** (↓ 30.0%)
Cora-ML	F1 (↑)	27.6 ± 1.6	39.0 ± 2.6	**35.0 ± 2.2** (↑ 26.8%)	**77.3 ± 1.3** (↑ 98.2%)
	Acc. (↑)	57.7 ± 0.4	62.8 ± 1.4	**59.0 ± 0.9** (↑ 2.3%)	**79.4 ± 0.6** (↑ 26.4%)
	KL. (↓)	3.6702	1.3419	**0.6314** (↓ 82.8%)	**0.0473** (↓ 96.5%)
CiteSeer	F1 (↑)	37.1 ± 1.7	56.9 ± 1.6	**41.9 ± 0.6** (↑ 12.9%)	**66.1 ± 1.2** (↑ 16.2%)
	Acc. (↑)	57.8 ± 0.6	65.4 ± 0.8	**59.6 ± 0.3** (↑ 3.1%)	**70.5 ± 1.0** (↑ 7.8%)
	KL. (↓)	0.5046	0.0418	**0.4107** (↓ 18.6%)	**0.0112** (↓ 73.2%)

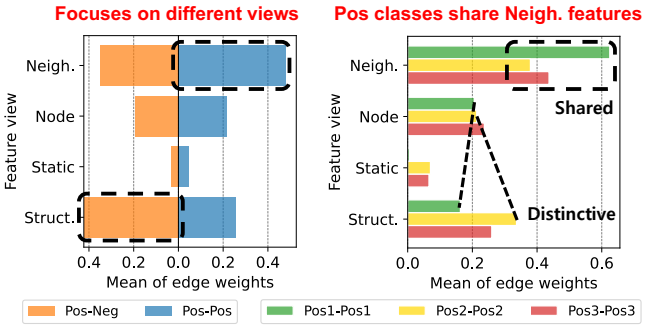

Figure 4: Average edge weights for each view, aggregated across positive-positive (Pos-Pos) and positive-negative (Pos-Neg) edges with unified positive classes (left), and averaged separately within each positive class (right). For a more detailed analysis, refer to Section 4.5.

as shown in the left plot of Figure 4. Note that D-MVP assigns high weights to the neighbor view for Pos-Pos edges, while emphasizing the structure view for Pos-Neg edges. This propagation strategy ensures that positive and negative nodes develop distinctive representations. This finding also highlights the importance of global features (structure view) when predicting negative nodes, which lack any observed labels.

Second, we analyze the edge weights for each positive class in the right plot of Figure 4. While positive classes share a common pattern of assigning high weights to the neighbor view (consistent with the left plot), their edge weights also exhibit distinctive patterns. For instance, Pos1-Pos1 edges have higher weights in the node view than in the structure view, whereas Pos2-Pos2 edges exhibit the opposite trend. These distinctive patterns among the feature blocks are critical for classifying different positive classes effectively.

4.6 Accuracy with Noisy Data (Q5)

For each iteration, D-MVP re-computes the edge weights for each feature view and propagates information based on these weights. From the experiments described in Sections 4.2-4.5, we demonstrated that D-MVP effectively enhances the quality of edge weights, which directly contributes to model performance. This raises an additional question: *"Can the edge re-weighting mechanism of D-MVP*

Table 4: Performance on noisy graphs with randomly added edges. The best and second-best performances are indicated in green and yellow. Note that D-MVP demonstrates superior performance to the baselines in most of the cases.

Models	Cora			Cora-ML			CiteSeer			CiteSeer-full			Chameleon		
	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)	F1 (↑)	Acc. (↑)	KL. (↓)
MLP	16.4 ± 0.0	48.3 ± 0.0	6.8919	17.3 ± 0.1	52.0 ± 0.0	4.3851	17.1 ± 0.1	51.2 ± 0.0	4.8054	24.3 ± 0.5	57.5 ± 0.2	1.0936	17.1 ± 0.2	46.5 ± 0.0	4.3415
GCN	19.6 ± 0.5	49.4 ± 0.2	1.6412	20.6 ± 1.4	53.0 ± 0.5	1.5928	20.3 ± 0.6	52.0 ± 0.2	1.3601	26.4 ± 0.4	58.2 ± 0.1	0.9543	20.2 ± 0.4	47.5 ± 0.2	1.5068
URE+MLP	16.7 ± 0.1	48.4 ± 0.0	4.0817	17.5 ± 0.0	52.1 ± 0.0	4.8231	17.2 ± 0.1	51.2 ± 0.0	5.6348	23.0 ± 1.4	57.0 ± 0.4	2.1126	17.0 ± 0.1	46.4 ± 0.0	2.1524
URE+GCN	17.3 ± 0.4	48.6 ± 0.1	5.7978	17.7 ± 0.4	52.2 ± 0.2	5.3835	18.5 ± 0.3	51.6 ± 0.1	5.0934	20.7 ± 1.5	56.4 ± 0.4	5.3027	15.8 ± 0.0	46.1 ± 0.0	8.3478
AREA+MLP	28.3 ± 1.9	52.1 ± 0.7	0.7519	27.7 ± 2.4	55.6 ± 0.8	0.7306	26.1 ± 2.6	53.1 ± 0.4	1.1807	53.7 ± 3.4	67.1 ± 2.4	0.0934	30.9 ± 2.0	51.4 ± 0.9	0.8221
AREA+GCN	57.6 ± 4.7	68.2 ± 1.9	0.0888	48.1 ± 4.5	65.6 ± 1.8	0.3265	56.2 ± 2.0	64.3 ± 2.2	0.0465	74.5 ± 3.6	81.5 ± 2.0	0.0259	25.7 ± 2.3	27.6 ± 2.0	4.2216
GRAB	53.2 ± 1.3	48.0 ± 0.2	8.2217	66.4 ± 18.1	63.9 ± 21.1	4.4542	41.6 ± 0.4	40.7 ± 0.5	8.7419	84.7 ± 1.4	85.8 ± 1.7	0.0212	36.8 ± 0.2	37.2 ± 0.3	7.8274
PU-GNN	80.1 ± 3.4	81.8 ± 3.2	0.0030	83.6 ± 1.2	84.7 ± 1.5	0.0083	65.8 ± 0.6	70.5 ± 1.0	0.0174	82.6 ± 0.6	86.0 ± 0.3	0.0027	36.8 ± 0.5	37.0 ± 0.6	7.8299
GPL	64.7 ± 0.9	66.6 ± 0.6	0.0224	34.8 ± 1.0	46.7 ± 0.6	2.0069	54.2 ± 0.3	62.2 ± 0.2	0.0631	75.0 ± 0.3	77.8 ± 0.2	0.0152	35.6 ± 0.2	48.7 ± 0.6	2.7251
D-MVP (Ours)	85.5 ± 0.5	86.0 ± 0.6	0.0030	85.7 ± 0.4	87.0 ± 0.4	0.0013	67.2 ± 0.5	71.4 ± 0.6	0.0304	84.8 ± 1.1	86.3 ± 1.2	0.0158	49.6 ± 2.7	60.8 ± 1.3	0.3687

Table 5: Running time of D-MVP and two recent graph-based PU methods: PU-GNN and GPL. D-MVP is significantly faster than PU-GNN and achieves comparable efficiency to GPL. Furthermore, the overhead of updating edge weights in the multi-view graph, which is performed only once per iteration, is negligible.

Run time (sec)	Cora	Cora-ML	CiteSeer	Cite-full	Cham.
PU-GNN	0.0304	0.0672	0.0279	0.0521	0.1748
GPL	0.0011	0.0015	0.0016	0.0016	0.0015
D-MVP (Ours)	0.0051	0.0062	0.0055	0.0063	0.0058
Edge reweighting	0.0010	0.0012	0.0012	0.0014	0.0013

also effectively handle noisy (non-real) edges?". To explore this, we conduct experiments on noisy graphs where edges are randomly added. This setting tests whether D-MVP can dynamically adjust edge weights to mitigate the impact of noisy edges.

Table 4 presents the performance of D-MVP and baselines on these noisy graphs, where the ratio r_{noise} of randomly added edges to the original edges is set to 0.1. Comparing the results in Tables 2 and 4, we observe that the accuracy of most baselines including GRAB (which shares the same loss function as D-MVP) declines significantly across the datasets. This failure occurs because these methods lack the ability of dynamically assigning edge weights considering the uncertainty of unlabeled nodes. In contrast, D-MVP significantly outperforms the baselines by learning edge weights for each view. This approach enables D-MVP to effectively capture both shared and distinctive features of positive classes, making it robust against noisy edges.

4.7 Training Efficiency (Q6)

We compare the per-epoch running time of D-MVP against two recent graph-based PU baselines. In addition, we report the time required to update edge weights in the multi-view graph of D-MVP, which is performed only once per iteration. The detailed results are presented in Table 5.

We observe that D-MVP trains significantly faster than PU-GNN and achieves comparable efficiency to GPL. Importantly, the cost of updating edge weights across multiple views—performed only once per iteration—introduces minimal overhead, demonstrating the practicality of the proposed multi-view propagation scheme.

5 CONCLUSION

We propose D-MVP, an accurate method for graph-based MPU learning. D-MVP effectively addresses the challenges of previous approaches, which are over-smoothing and the lack of common properties in positive classes, by disentangling feature propagation into multiple views. D-MVP assigns distinct weights to each view and aggregates information differently, enabling the node classifier to capture both shared and distinctive features among positive classes. By analyzing the learned weights, D-MVP provides interpretability by revealing how different feature views contribute to class-specific representations. Extensive experiments on real-world datasets show that D-MVP consistently outperforms existing methods, highlighting its effectiveness and robustness in graph-based MPU learning.

ACKNOWLEDGMENTS

This work was supported by the National Research Foundation of Korea (NRF) funded by MSIT (2022R1A2C3007921). This work was also supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [No.2022-0-00641, XVoice: Multi-Modal Voice Meta Learning], [No.RS-2024-00509257, Global AI Frontier Lab], [No.RS-2021-II211343, Artificial Intelligence Graduate School Program (Seoul National University)], and [No.RS-2021-II212068, Artificial Intelligence Innovation Hub (Artificial Intelligence Institute, Seoul National University)]. The Institute of Engineering Research at Seoul National University and the ICT at Seoul National University provided research facilities for this work. U Kang is the corresponding author.

REFERENCES

[1] Atin Basuchoudhary, Mohamed Eltoweissy, Mohamed Azab, Laura Razzolini, and Shimaa Mohamed. 2015. Cyberdefense when attackers mimic legitimate users: a Bayesian approach. In *Information Reuse and Integration for Data Science.*
[2] Alex Bavelas. 1950. Communication patterns in task-oriented groups. *The journal of the acoustical society of America.*
[3] Yoshua Bengio, Aaron Courville, and Pascal Vincent. 2013. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence.*
[4] Aleksandar Bojchevski and Stephan Günnemann. 2018. Deep gaussian embedding of graphs: Unsupervised inductive learning via ranking. In *International Conference on Learning Representations (ICLR).*
[5] Phillip Bonacich. 1972. Factoring and weighting approaches to status scores and clique identification. *Journal of mathematical sociology.*
[6] Marthinus Christoffel du Plessis, Gang Niu, and Masashi Sugiyama. 2014. Analysis of Learning from Positive and Unlabeled Data. In *NeurIPS.*

[7] Charles Elkan and Keith Noto. 2008. Learning classifiers from only positive and unlabeled data. In *SIGKDD International Conference on Knowledge Discovery and Data Mining*.

[8] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *International Conference on Learning Representations (ICLR)*.

[9] LC Freeman. 1977. A set of measures of centrality based on betweenness. *Sociometry*.

[10] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *SIGKDD International Conference on Knowledge Discovery and Data Mining*.

[11] Simon Haykin. 1994. *Neural networks: a comprehensive foundation*. Prentice Hall PTR.

[12] Irina Higgins, Loic Matthey, Arka Pal, Christopher P Burgess, Xavier Glorot, Matthew M Botvinick, Shakir Mohamed, and Alexander Lerchner. 2017. beta-vae: Learning basic visual concepts with a constrained variational framework. *International Conference on Learning Representations (ICLR)*.

[13] Yu-Guan Hsieh, Gang Niu, and Masashi Sugiyama. 2019. Classification from Positive, Unlabeled and Biased Negative Data. In *ICML*.

[14] Jinhong Jung, Woojeong Jin, Ha-myung Park, and U Kang. 2021. Accurate relational reasoning in edge-labeled graphs by multi-labeled random walk with restart. *World Wide Web*.

[15] Abedin Keshavarz and Amir Lakizadeh. 2024. PU-GNN: A Positive-Unlabeled Learning Method for Polypharmacy Side-Effects Detection Based on Graph Neural Networks. *International Journal of Intelligent Systems*.

[16] Junghun Kim, Jinhong Jung, and U Kang. 2021. Compressing deep graph convolution network with multi-staged knowledge distillation. *Plos one*.

[17] Junghun Kim, Ka Hyun Park, Hoyoung Yoon, and U Kang. 2024. Domain-Aware Data Selection for Speech Classification via Meta-Reweighting. In *Proc. Interspeech 2024*. 797–801.

[18] Junghun Kim, Ka Hyun Park, Hoyoung Yoon, and U Kang. 2025. Accurate Link Prediction for Edge-Incomplete Graphs via PU Learning. In *AAAI*.

[19] Minjun Kim, Jaehyeon Choi, SeungJoo Lee, Jinhong Jung, and U Kang. 2025. AugWard: Augmentation-Aware Representation Learning for Accurate Graph Classification. *PAKDD*.

[20] Diederik P Kingma. 2025. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

[21] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.

[22] Ryuichi Kiryo, Gang Niu, Marthinus Christoffel du Plessis, and Masashi Sugiyama. 2017. Positive-Unlabeled Learning with Non-Negative Risk Estimator. In *NeurIPS*.

[23] Daphane Koller. 2009. Probabilistic Graphical Models: Principles and Techniques.

[24] SeungJoo Lee, Yong-Chan Park, and U Kang. 2024. Accurate Coupled Tensor Factorization with Knowledge Graph. In *2024 IEEE International Conference on Big Data (BigData)*.

[25] Fabien Letouzey, François Denis, and Régis Gilleron. 2000. Learning from positive and unlabeled examples. In *ALT*.

[26] Wen-Sheng Li, Bing Lee, and Kwok-Shing Leung. 2009. Learning from positive and unlabelled examples using maximum margin clustering. *IJDWM*.

[27] Xiaoli Li and Bing Liu. 2003. Learning to classify texts using positive and unlabeled data. In *IJCAI*.

[28] Zhongnian Li, Meng Wei, Peng Ying, Tongfeng Sun, and Xinzheng Xu. 2024. Learning from Concealed Labels. In *Multimedia*.

[29] Zhongnian Li, Meng Wei, Peng Ying, and Xinzheng Xu. 2024. ESA: Example Sieve Approach for Multi-Positive and Unlabeled Learning. *arXiv preprint arXiv:2412.02240*.

[30] Yating Lin, Xiaoqi Chen, Yuxiang Lin, Xu Xiao, Zhibin Huang, Wenxian Yang, Rongshan Yu, and Jiahuai Han. 2024. Accurate Cell Abundance Quantification using Multi-positive and Unlabeled Self-learning. *bioRxiv*.

[31] Bing Liu, Wee Sun Lee, Philip S Yu, and Xiaoli Li. 2002. Partially supervised classification of text documents. In *ICML*.

[32] Fan Liu, Huilin Chen, Zhiyong Cheng, Anan Liu, Liqiang Nie, and Mohan Kankanhalli. 2022. Disentangled multimodal representation learning for recommendation. *IEEE Transactions on Multimedia*.

[33] Yanbei Liu, Xiao Wang, Shu Wu, and Zhitao Xiao. 2020. Independence Promoted Graph Disentangled Networks. In *AAAI*.

[34] Zewen Liu, Guancheng Wan, B Aditya Prakash, Max SY Lau, and Wei Jin. 2024. A review of graph neural networks in epidemic modeling. In *SIGKDD International Conference on Knowledge Discovery and Data Mining*.

[35] Jianxin Ma, Peng Cui, Kun Kuang, Xin Wang, and Wenwu Zhu. 2019. Disentangled graph convolutional networks. In *ICML*.

[36] Shuangxun Ma and Ruisheng Zhang. 2017. PU-LP: A novel approach for positive and unlabeled learning by label propagation. In *International Conference on Multimedia and Expo*.

[37] Yujie Mo, Yajie Lei, Jialie Shen, Xiaoshuang Shi, Heng Tao Shen, and Xiaofeng Zhu. 2023. Disentangled multiplex graph representation learning. In *ICML*. PMLR.

[38] Gang Niu, Marthinus Christoffel du Plessis, Tomoya Sakai, Yao Ma, and Masashi Sugiyama. 2016. Theoretical Comparisons of Positive-Unlabeled Learning against Positive-Negative Learning. In *NeurIPS*.

[39] Curtis Northcutt, Lu Jiang, and Isaac Chuang. 2017. Learning with confident examples: Rank pruning for robust classification with noisy labels. In *ICML*.

[40] Lawrence Page. 1999. *The PageRank citation ranking: Bringing order to the web*. Technical Report. Technical Report.

[41] Cheong Hee Park. 2022. Multi-Class Positive and Unlabeled Learning for High Dimensional Data Based on Outlier Detection in a Low Dimensional Embedding Space. *Electronics*.

[42] Giorgio Patrini, Frank Nielsen, Richard Nock, and Marcello Carioni. 2016. Loss factorization, weakly supervised learning and label noise robustness. In *ICML*.

[43] Amedeo Racanati, Roberto Esposito, and Dino Ienco. 2022. Dealing With Multi-positive Unlabeled Learning Combining Metric Learning and Deep Clustering. *IEEE Access*.

[44] T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra. 2023. A survey on oversmoothing in graph neural networks. *arXiv*.

[45] Stephen B Seidman. 1983. Network structure and minimum degree. *Social networks*.

[46] Sooyeon Shim, Junghun Kim, Kahyun Park, and U Kang. 2024. Accurate graph classification via two-staged contrastive curriculum learning. *Plos one*.

[47] Senlin Shu, Zhuoyi Lin, Yan Yan, and Li Li. 2020. Learning from multi-class positive and unlabeled data. In *International Conference on Data Mining (ICDM)*.

[48] Xin Wang, Hong Chen, Yuwei Zhou, Jianxin Ma, and Wenwu Zhu. 2022. Disentangled representation learning for recommendation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

[49] Xinrui Wang, Wenhai Wan, Chuanxing Geng, Shaoyuan Li, and Songcan Chen. 2023. Beyond Myopia: Learning from Positive and Unlabeled Data through Holistic Predictive Trends. In *NeurIPS*.

[50] Yifan Wang, Xiao Luo, Chong Chen, Xian-Sheng Hua, Ming Zhang, and Wei Ju. 2024. DisenSemi: Semi-supervised graph classification via disentangled representation learning. *IEEE Transactions on Neural Networks and Learning Systems*.

[51] Duncan J Watts and Steven H Strogatz. 1998. Collective dynamics of 'small-world'networks. *Nature*.

[52] Man Wu, Shirui Pan, Lan Du, Ivor W. Tsang, Xingquan Zhu, and Bo Du. 2019. Long-short Distance Aggregation Networks for Positive Unlabeled Graph Learning. In *International Conference on Information and Knowledge Management (CIKM)*.

[53] Yuhao Wu, Jiangchao Yao, Bo Han, Lina Yao, and Tongliang Liu. 2024. Unraveling the Impact of Heterophilic Structures on Graph Positive-Unlabeled Learning. *ICML*.

[54] Yixing Xu, Chang Xu, Chao Xu, and Dacheng Tao. 2017. Multi-Positive and Unlabeled Learning.. In *IJCAI*.

[55] Hansi Yang, Yongqi Zhang, Quanming Yao, and James T. Kwok. 2023. Positive-Unlabeled Node Classification with Structure-aware Graph Learning. In *International Conference on Information and Knowledge Management (CIKM)*.

[56] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *ICML*.

[57] Jaemin Yoo, Hyunsik Jeon, Jinhong Jung, and U Kang. 2022. Accurate node feature estimation with structured variational graph autoencoder. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.

[58] Jaemin Yoo, Junghun Kim, Hoyoung Yoon, Geonsoo Kim, Changwon Jang, and U Kang. 2021. Accurate graph-based PU learning without class prior. In *International Conference on Data Mining (ICDM)*.

[59] Jaemin Yoo, Junghun Kim, Hoyoung Yoon, Geonsoo Kim, Changwon Jang, and U Kang. 2022. Graph-based PU learning for binary and multiclass classification without class prior. *Knowledge and Information Systems*.

[60] Jaemin Yoo, Meng-Chieh Lee, Shubhranshu Shekhar, and Christos Faloutsos. 2023. Less is more: Slimg for accurate, robust, and interpretable graph mining. In *SIGKDD International Conference on Knowledge Discovery and Data Mining*.

[61] Ke Zhang, Chenxi Zhang, Xin Peng, and Chaofeng Sha. 2022. Putracead: Trace anomaly detection with partial labels based on GNN and Pu Learning. In *International Symposium on Software Reliability Engineering*.

[62] Zeyang Zhang, Xin Wang, Ziwei Zhang, Guangyao Shen, Shiqi Shen, and Wenwu Zhu. 2024. Unsupervised graph neural architecture search with disentangled self-supervision. *NeurIPS*.

[63] Yunrui Zhao, Qianqian Xu, Yangbangyan Jiang, Peisong Wen, and Qingming Huang. 2022. Dist-PU: Positive-Unlabeled Learning from a Label Distribution Perspective. In *CVPR*.

[64] Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. 2004. Learning with local and global consistency. *NeurIPS*.

[65] Kang Zhou, Yuepei Li, and Qi Li. 2022. Distantly Supervised Named Entity Recognition via Confidence-Based Multi-Class Positive and Unlabeled Learning. In *ACL (1)*.

A APPENDIX

A.1 Symbols

The symbols used in this paper is summarized in Table 6.

Table 6: Symbols.

Symbol	Description
$G(\mathcal{V}, \mathcal{E})$	Undirected graph consisting of $\mathcal{V}$ and $\mathcal{E}$
$\mathcal{V}, \mathcal{E}$	Sets of all nodes and edges, respectively
$\mathcal{P}_m$	Set of labeled nodes of m-th positive class
$\mathcal{U}$	Set of unlabeled nodes that we aim to classify
$\mathbf{X}$	Feature matrix ($\in \mathbb{R}^{N \times F_{\text{in}}}$) for all nodes in G
$\mathbf{y}$	Label vector for the labeled nodes in $\mathcal{P}_m$ for $m = 1, \ldots, M$
N	Number of nodes ($= \lvert\mathcal{V}\rvert$)
F_{in}	Number of input features for each node
M	Number of positive classes
R	Number of feature views of D-MVP

A.2 Detailed Settings of Experiments

We provide detailed implementation settings for D-MVP and the baselines. For the baselines that use GCN or MLP as the backbone model, we set the numbers of layers and units as 2 and 16, respectively, following the conventional settings [53, 55, 59]. We train all the models using the Adam optimizer [20] with 500 epochs. For URE, AREA, and GPL, we extend the training to 2,000 epochs as we observe significant fluctuations in the loss during earlier epochs. All experiments are conducted on a single machine equipped with an NVIDIA GTX 1080 Ti GPU.

MLP [11]. We train a Multi-Layer Perceptron (MLP) using the cross-entropy loss function, treating all unlabeled nodes as negatives.

GCN [21]. We utilize the Graph Convolutional Network (GCN) implementation from torch-geometric package [8]. The model is trained with the cross-entropy loss function while treating the unlabeled nodes as negatives.

URE [54]. URE uses a loss function which generalizes the unbiased risk estimator [6] to MPU learning. Since no official implementation is available, we reimplement URE. We use MLP and GCN as backbone models, reporting results for both configurations (URE+MLP and URE+GCN).

AREA [47]. AREA uses an alternative risk estimator designed to prevent overfitting by ensuring the risk remains non-negative. We reimplement AREA since there is no public implementation from the authors. We adopt MLP and GCN as backbone models, and report the results for both configurations (AREA+MLP and AREA+GCN).

GRAB [59]. GRAB uses an iterative belief-based loss function that does not require prior knowledge of class distribution. We reimplement GRAB due to the absence of a public implementation provided by the authors. For early stopping at the iteration level, we set the patience to 1, promoting more stable learning.

PU-GNN [55]. PU-GNN introduces a distance-aware loss that leverages graph homophily to provide more reliable supervision in graph-based MPU (and PU) learning. As no public implementation is available, we reimplement PU-GNN. Additionally, we omit the structural regularization loss term, as we find it to negatively impact performance in graph-based MPU learning.

GPL [53]. GPL reduces heterophily in graphs by performing bilevel optimization during the training of the node classifier. Since an official implementation is unavailable, we reimplement GPL.

D-MVP (proposed). D-MVP exploits multi-view features that incorporate both local and global topological information, specifically neighbor and structural features. This allows D-MVP to perform effectively without requiring multiple stacked propagation layers. Thus, we set the number of layers as 1 while allocating 16 hidden dimensions for each view. For early stopping at the iteration level (outer loop in Algorithm 1), we set the patience to 1, ensuring more stable learning.